

7.03 Algorithms

7.03a	Definition of an algorithm	a) Understand that an algorithm has an input and an output, is deterministic and finite. <i>Includes the use of a counter and the use of a stopping condition in an algorithm.</i> <i>Be familiar with the terms “greedy”, “heuristic” and “recursive” in the context of algorithms.</i>	
7.03b	Awareness of the uses and practical limitations of algorithms	b) Appreciate why an algorithmic approach to problem-solving is generally preferable to ad hoc methods, and understand the limitations of algorithmic methods. <i>Includes understanding that algorithmic methods are used by computers for solving large scale problems and that small scale problems are only being used to demonstrate how a given algorithm works.</i>	

OCR Ref.	Subject Content	Stage 1 learners should...	Stage 2 learners should additionally...
7.03c	Working with algorithms	c) Trace through an algorithm and interpret what the algorithm has achieved. Algorithms may be presented as flow diagrams, listed in words, or written in simple pseudo-code. <i>Includes understanding and being able to use the functions $\text{INT}(x)$ and $\text{ABS}(x)$. Learners may find it useful to have a calculator with these functions, but large numbers of repeated applications will not be required in the assessment.</i> <i>Includes adapting or altering an algorithm to achieve a given purpose, and adjusting a short set of instructions to create an algorithm.</i> <i>[Programming skills will not be required.]</i>	
7.03d	The order of an algorithm	d) Use the order of an algorithm to calculate an approximate run-time for a large problem by scaling up a given run-time. <i>Includes understanding that when the “maximum run-time” of an algorithm is represented as a function of the “size” of the problem, the order of the algorithm, for very large sized problems, is given by the dominant term.</i> <i>Learners should know that the sum of the first n positive integers is $\sum_{r=1}^n r = \frac{1}{2}n(n+1)$.</i> <i>Learners should be familiar with the notation $O(n^4)$ and the concept of dominance in an informal sense only.</i>	

OCR Ref.	Subject Content	Stage 1 learners should...	Stage 2 learners should additionally...
7.03e	Efficiency and complexity	e) Compare the efficiency of two algorithms that achieve the same end result by considering a given aspect of the run-time in a specific case. <i>e.g. The number of swaps or comparisons to sort a given list.</i>	h) Calculate the run-time as a function of the size by considering the best case, the worst case or <i>Includes considering all cases and averaging where a</i> i) Be familiar with: $O(n^k)$ for $k \in \mathbb{R}$, $O(a^n)$ for $a > 0$, $O(\log n)$ where n is a measure of the size of th <i>Know the hierarchy of orders and what this means in efficiency. Learners should be aware that:</i> $O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(a^n) \subset O(n!)$, which is given in the Formulae Boo.
7.03f 7.03h		f) Calculate worst case time complexity, the “maximum run-time” $T(n)$, as a function of the size of a problem by considering the worst case for a specific problem. <i>Includes cases of the algorithms for sorting and standard network problems studied in this specification.</i> <i>Includes an informal understanding that, for example $T(n) = n^2 + n^4$ is order n^4, or equivalently $O(n^4)$.</i>	
7.03g 7.03i		g) Be familiar with $O(n^k)$, where n is a measure of the size of the problems and $k = 0, 1, 2, 3$ or 4.	

OCR Ref.	Subject Content	Stage 1 learners should...	Stage 2 learners should additionally...
7.03j 7.03k	Strategies for sorting	j) Be able to sort a list using bubble sort and using shuttle sort. <i>Bubble sort and shuttle sort will start at the left-hand end of the list, unless specified otherwise in the question.</i> <i>Includes knowing that, in general, sorting algorithms have quadratic order as a function of the length of the list.</i>	k) Be able to sort a list using quick sort. <i>Quick sort will pivot on the first value, unless specified otherwise in a question.</i> <i>Includes knowing that quick sort is only $O(n^2)$ in the worst case.</i> <i>Questions may be set that interrogate the application of the method, for example whether the choice of pivot affects the efficiency of quick sort.</i>
7.03l 7.03m	Strategies for packing	l) Be familiar with the next-fit, first-fit, first-fit decreasing and full bin methods for one-dimensional packing problems. <i>Includes knowing that these are heuristic algorithms.</i> <i>Includes the terms “online” and “offline”.</i>	m) Extend their knowledge of packing methods. <i>e.g. Packing problems in two or three dimensions.</i> <i>e.g. Knapsack problems: given a set of items, each with a mass and a profit, determine which to include so that the total mass does not exceed some given limit and the total profit is as large as possible.</i>